

LAPORAN PRAKTIKUM 2
PEMOGRAMAN BERORIENTASI OBJEK

“RELASI ANTAR OBJEK”

DISUSUN OLEH :

DZHILLAN DZHALILA

2511531001

PRAKTIKUM KELAS C

DOSEN PENGAMPU:

NURFIAH, S.ST, M.KOM



FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2026

1.1 LANGKAH KERJA PRAKTIKUM

1.1.1 Membuat Program Transaksi

- a. Buat file Transaksi.java di package Pekan2. Class ini jadi model untuk satu catatan transaksi (setor/tarik) yang nanti dipakai oleh Rekening2.
- b. Tambahkan atribut:
idTransaksi (String) untuk identitas unik tiap transaksi
jenis (String) menandai jenis transaksi, misalnya "Kredit" (setor) atau "Debit" (tarik)
nominal (double) jumlah uang pada transaksi tersebut
- c. Buat constructor Transaksi(String id, String jenis, double nominal) untuk menginisialisasi ketiga atribut di atas saat objek Transaksi dibuat. Constructor ini dipanggil dari dalam setorTunai() dan tarikTunai() milik Rekening2.
- d. Tambahkan method getter:
getJenis() mengembalikan nilai jenis
getNominal() mengembalikan nilai nominal
Ini menerapkan prinsip encapsulation, field bisa dibaca dari luar class tanpa akses langsung.
- e. Buat method cetakDetail() untuk menampilkan satu baris ringkasan transaksi (ID, jenis, nominal). Method ini dipanggil dari Rekening2.cekRiwayatTransaksi() untuk setiap item di ArrayList Transaksi.
- f. Uji integrasi, pastikan Rekening2 bisa membuat objek Transaksi baru tiap kali setorTunai() atau tarikTunai() dipanggil, menyimpannya ke riwayatTransaksi, dan cetakDetail() tampil benar saat menu Cek Riwayat Transaksi dipilih di Main2.

HASIL PROGRAM

```
package Pekan2;  
  
public class Transaksi {  
    String idTransaksi;  
    String jenis;
```

```

double nominal;

public String getJenis() {
    return jenis;
}

public double getNominal() {
    return nominal;
}

public Transaksi(String id, String jenis, double nominal) {
    this.idTransaksi = id;
    this.jenis = jenis;
    this.nominal = nominal;
}

public void cetakDetail () {
    System.out.println("ID: " + idTransaksi + " | Jenis : "+
jenis + " | Nominal : Rp" + nominal);
}
}

```

1.1.2 Membuat Program Rekening2

- a. Buat package baru Pekan2, lalu buat file Rekening2.java di dalamnya.
- b. Buat class Transaksi terlebih dahulu (di package yang sama), karena Rekening2 bergantung pada class ini. Minimal Transaksi punya atribut seperti idTransaksi, jenis ("Kredit"/"Debit"), nominal, dan method cetakDetail() untuk menampilkan detail satu transaksi.
- c. Definisikan state/atribut di Rekening2: nomorRekening, namaPemilik, saldo — sama seperti sebelumnya, ditambah satu atribut baru: ArrayList<Transaksi> riwayatTransaksi (relasi 1-to-many antara satu rekening dan banyak transaksi).
- d. Buat constructor Rekening2(String nomor, String nama, double saldoAwal):
- e. Inisialisasi nomorRekening, namaPemilik, saldo seperti biasa.
- f. Wajib inisialisasi riwayatTransaksi = new ArrayList<>(); di dalam constructor : jikalau lupa, akan muncul NullPointerException saat mencoba .add().
- g. Buat method setorTunai(double nominal):

- i) Validasi nominal > 0 .
 - ii) Tambah ke saldo, lalu buat objek Transaksi baru (id unik pakai `System.currentTimeMillis()`, jenis "Kredit", nominal) dan `add()` ke `riwayatTransaksi`.
- h. Buat method `tarikTunai(double nominal)`:
 - i) Validasi saldo cukup dan nominal minimal Rp10.000.
 - ii) Kalau valid, kurangi saldo, buat objek Transaksi baru (jenis "Debit"), lalu simpan ke `riwayatTransaksi`.
- i. Buat method `cekInformasi()` : menampilkan ringkasan rekening (sama seperti versi sebelumnya).
- j. Buat method `cekRiwayatTransaksi()` : melakukan iterasi (for-each) ke seluruh isi `riwayatTransaksi` dan memanggil `t.cetakDetail()` untuk tiap objek Transaksi, atau menampilkan pesan kalau list masih kosong.
- k. Update `Main.java` di package `Pekan2` supaya menggunakan `Rekening2` (bukan `Rekening`), dan tambahkan menu baru untuk memanggil `cekRiwayatTransaksi()`.
- l. Jalankan dan uji: buka rekening → setor beberapa kali → tarik tunai → cek riwayat transaksi, pastikan semua transaksi tercatat dengan jenis dan nominal yang benar.

Hasil Kode Program :

```
package Pekan2;
import java.util.*;

public class Rekening2 {

    String nomorRekening;
    String namaPemilik;
    double saldo;

    //implementasi asosiasi (1-to-many)
    ArrayList <Transaksi> riwayatTransaksi;

    public Rekening2 (String nomor, String nama, double saldoAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        //wajib menginisiasi arraylist di dalam constructor
        agar tidak nullPointerException
    }
}
```

```

        this.riwayatTransaksi = new ArrayList <> ();

        System.out.println("Rekening atas nama " +
namaPemilik + " berhasil dibuat dengan saldo awal : Rp" +
saldo);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            String idTrx = "TRX-S-" +
System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi
(idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Setor tunai Rp" +
nominal + " berhasil. Saldo saat ini : Rp" + saldo);
        } else {
            System.out.println("Gagal : Nominal
setor harus lebih dari 0!");
        }
    }
}

```

1.1.3 Membuat Class Main2

- a. Buat file Main2.java di package Pekan2, import java.util.ArrayList dan java.util.Scanner.
- b. Siapkan struktur data multi-akun: buat ArrayList<Rekening2> daftarRekening untuk menyimpan semua rekening yang pernah dibuka, dan Rekening2 akunAktif = null untuk menandai rekening yang sedang aktif dipakai.
- c. Buat loop utama while (isRunning) yang menampilkan menu, termasuk baris status "Akun Aktif saat ini" — pakai ternary untuk menampilkan "-" jika belum ada akun aktif, atau nomor + nama pemilik jika sudah ada.
- d. Tambahkan validasi input angka sebelum membaca pilihan menu: cek input.hasNextInt() : kalau bukan angka, buang baris input yang salah (input.nextLine()) dan continue ke iterasi loop berikutnya (mencegah program crash karena InputMismatchException).
- e. Case 1 : Buka Rekening Baru:
 - i) Minta nomor rekening.

- ii) Cek duplikasi: iterasi daftarRekening dengan for-each, bandingkan nomorRekening pakai equalsIgnoreCase. Jika sudah ada → tampilkan error dan break (batal buat rekening).
 - iii) Kalau belum ada, lanjut minta nama pemilik dan saldo awal (dengan validasi hasNextDouble() juga).
 - iv) Buat objek Rekening2 baru, add() ke daftarRekening, lalu langsung jadikan sebagai akunAktif.
- f. Case 2 : Setor Tunai: cek akunAktif tidak null → validasi input nominal berupa angka → panggil akunAktif.setorTunai(setor).
- g. Case 3 : Tarik Tunai: sama seperti Setor Tunai, tapi memanggil akunAktif.tarikTunai(tarik).
- h. Case 4 : Cek Informasi Rekening: cek akunAktif tidak null → panggil akunAktif.cekInformasi().
- j. Case 5 : Ganti Akun (fitur baru):
 - i) Cek dulu apakah daftarRekening kosong → kalau kosong, tampilkan error dan break.
 - ii) Minta nomor rekening yang dicari.
 - iii) Iterasi daftarRekening untuk mencari objek dengan nomorRekening yang cocok (equalsIgnoreCase), simpan ke variabel ditemukan.
 - iv) Jika ditemukan → set sebagai akunAktif baru dan tampilkan pesan sukses. Jika tidak → tampilkan error "tidak ditemukan".
- i. Case 6 : Cek Riwayat Transaksi: cek akunAktif tidak null → panggil akunAktif.cekRiwayatTransaksi().
- j. Case 0 : Keluar: set isRunning = false untuk menghentikan loop.
- k. Default: tampilkan pesan pilihan tidak valid.
- l. Tutup input.close() setelah loop selesai.
- m. Jalankan dan uji seluruh alur: buka beberapa rekening berbeda → coba buka dengan nomor yang sama (harus


```

        akunAktif.setorTunai(setor); //
Memanggil Behavior / method
    }
    break;

    case 3:
        System.out.println("Fitur ini akan
dikerjakan sebagai Tugas Mandiri.");
        break;

    case 4:
        if (akunAktif == null) {
            System.out.println("Error: Anda
belum membuka rekening!");
        } else {
            akunAktif.cekInformasi();
        }
        break;

    case 0:
        isRunning = false;
        System.out.println("Sistem ditutup.
Terima kasih!");
        break;

    default:
        System.out.println("Pilihan tidak
valid!");
    }
}
input.close();
}
}
}

```

1.2 TUGAS/LATIHAN PRAKTIKUM

1.2.1 Program Main2.java

```

package Pekan1;

import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // Tugas 2: gunakan Collection untuk
        menyimpan banyak rekening
        ArrayList<Rekening> daftarRekening = new
        ArrayList<>();
        Rekening akunAktif = null; // rekening yang
        sedang dipakai transaksi
    }
}

```

```

        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI
===");

        while (isRunning) {
            System.out.println("\nMenu Utama:");
            System.out.println("Akun Aktif saat
ini: " + (akunAktif == null ? "-" :
akunAktif.nomorRekening + " (" + akunAktif.namaPemilik +
")"));
            System.out.println("1. Buka Rekening
Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi
Rekening");
            System.out.println("5. Ganti Akun");
            System.out.println("6. Cek Riwayat
Transaksi (Mutasi)");
            System.out.println("7. Challenge 3");
            System.out.println("8. Tampilkan 3
Riwayat terbaru Challenge 2");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");

            if (!input.hasNextInt()) {
                System.out.println("Error :
Input harus berupa angka");
                input.nextLine();
                continue;
            }

            int pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan No
Rekening: ");

                    String no = input.nextLine();

                    boolean sudahAda = false;
                    for (Rekening r :
daftarRekening) {
                        if
(r.nomorRekening.equalsIgnoreCase(no)) {
                            sudahAda = true;
                            break;
                        }
                    }
                    if (sudahAda) {

                        System.out.println("Error : Nomor rekening sudah
terdaftar!");
                    }
                }
            }
        }
    }
}

```

```

                break;
            }

            System.out.print("Masukkan Nama
Pemilik: ");
            String nama = input.nextLine();

            System.out.print("Masukkan Saldo
Awal : ");
            if (!input.hasNextDouble()) {
                System.out.println("Error : Input harus berupa
angka");
                input.nextLine();
                break;
            }
            double saldo =
input.nextDouble();
            input.nextLine();

            Rekening rekeningBaru = new
Rekening(no, nama, saldo);

            daftarRekening.add(rekeningBaru);
            akunAktif = rekeningBaru;
            break;

            case 2:
                if (akunAktif == null) {

                    System.out.println("Error : Mohon maaf, Anda belum
memiliki nomor rekening!");
                } else {
                    System.out.print("Masukkan
nominal setor : ");
                    if
(!input.hasNextDouble()) {
                        System.out.println("Error : Input harus berupa
angka");
                        input.nextLine();
                        break;
                    }
                    double setor =
input.nextDouble();
                    input.nextLine();

                    akunAktif.setorTunai(setor);
                }
                break;

            case 3:
                if (akunAktif == null) {

                    System.out.println("Error : Mohon maaf, Anda belum
memiliki nomor rekening!");
                } else {

```

```

        System.out.print("Masukkan
nominal tarik : ");
        if
(!input.hasNextDouble()) {
            System.out.println("Error : Input harus berupa
angka");
            input.nextLine();
            break;
        }
        double tarik =
input.nextDouble();
        input.nextLine();

        akunAktif.tarikTunai(tarik);
        }
        break;

        case 4:
            if (akunAktif == null) {

                System.out.println("Error : Anda belum membuka
rekening");
            } else {
                akunAktif.cekInformasi();
            }
            break;

        case 5:
            if (daftarRekening.isEmpty()) {
                System.out.println("Error : Belum ada
rekening yang terdaftar!");
            }
            break;
            System.out.print("Masukkan No Rekening yang
ingin diaktifkan: ");
            String cariNo = input.nextLine();

            Rekening2 ditemukan = null;
            for (Rekening2 r : daftarRekening) {
                if
(r.nomorRekening.equalsIgnoreCase(cariNo)) {
                    ditemukan = r;
                    break;
                }
            }

            if (ditemukan != null) {
                akunAktif = ditemukan;
                System.out.println("Berhasil beralih ke
rekening " + akunAktif.nomorRekening + " (" + akunAktif.namaPemilik +
")");
            } else {
                System.out.println("Error : Nomor
rekening tidak ditemukan!");
            }
            break;

```

```

        case 6 :
            if (akunAktif == null) {
                System.out.println("Error : Anda belum
membuka rekening");
            } else {
                akunAktif.cekRiwayatTransaksi();
            }
            break;

        case 7 :
            if (akunAktif == null) {
                System.out.println("Error : Anda belum
membuka rekening");
            } else {
                akunAktif.challenge3();
            }
            break;

        case 8 :
            if (akunAktif == null) {
                System.out.println("Error : Anda belum
membuka rekening");
            } else {
                akunAktif.challenge2();
            }
            break;

        case 0:
            isRunning = false;
            System.out.println("Sistem ditutup. Terima
kasih!");
            break;

        default:
            System.out.println("Pilihan tidak valid!");
    }
    input.close();
}
}

```

1.2.2 Program Rekening2.java

```

package Pekan2;
import java.util.*;

public class Rekening2 {

    String nomorRekening;
    String namaPemilik;
    double saldo;

    //implementasi asosiasi (1-to-many)
    ArrayList <Transaksi> riwayatTransaksi;
}

```

```

public Rekening2 (String nomor, String nama, double saldoAwal) {
    this.nomorRekening = nomor;
    this.namaPemilik = nama;
    this.saldo = saldoAwal;

    //wajib menginisiasi arraylist di dalam constructor agar tidak
    nullPointerException
    this.riwayatTransaksi = new ArrayList <> ();

    System.out.println("Rekening atas nama " + namaPemilik + "
    berhasil dibuat dengan saldo awal : Rp" + saldo);
}

public void setorTunai(double nominal) {
    if (nominal > 0) {
        saldo += nominal;
        String idTrx = "TRX-S-" +
        System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi (idTrx, "Kredit",
        nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Setor tunai Rp" + nominal + "
        berhasil. Saldo saat ini : Rp" + saldo);
    } else {
        System.out.println("Gagal : Nominal setor harus
        lebih dari 0!");
    }
}

public void tarikTunai (double nominal) {
    if (nominal > saldo ) {
        System.out.println("Transaksi Gagal : Saldo Anda tidak
        cukup. Saldo Anda : Rp" + saldo);
    } else if (nominal < 10000) {
        System.out.println("Transaksi Gagal : Minimal transaksi
        tarik tunai adalah sebesar Rp10000");
    } else {
        saldo -= nominal;
        String idTrx = "TRX-T-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi (idTrx, "Debit" ,
        nominal);
        riwayatTransaksi.add(trxBaru);
        System.out.println("Tarik tunai Rp" + nominal + "
        berhasil. Saldo saat ini : Rp" + saldo);
    }
}

public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.print("Saldo Akhir : Rp" + saldo);
    System.out.println();
    System.out.println("-----");
}

```

```

public void cekRiwayatTransaksi() {
    System.out.println("--- RIWAYAT TRANSAKSI ---");
    System.out.println("No. Rekening : " + nomorRekening);
    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Belum ada transaksi.");
    } else {
        for (Transaksi t : riwayatTransaksi) {
            t.cetakDetail();
        }
    }
    System.out.println("-----");
}

public void challenge3() {
    double totalSetor = 0;
    double totalTarik = 0;

    for (Transaksi t : riwayatTransaksi) {
        if (t.getJenis().equalsIgnoreCase("Kredit")) {
            totalSetor += t.getNominal();
        } else if (t.getJenis().equalsIgnoreCase("Debit")) {
            totalTarik += t.getNominal();
        }
    }

    double akumulasi = totalSetor - totalTarik;

    System.out.println("--- RINGKASAN TRANSAKSI ---");
    System.out.println("No. Rekening      : " +
nomorRekening);
    System.out.println("Total Setor          : Rp" + totalSetor);
    System.out.println("Total Tarik          : Rp" + totalTarik);
    System.out.println("Akumulasi Transaksi  : Rp" + akumulasi);
    System.out.println("Saldo Akhir          : Rp" + saldo);
    System.out.println("-----");
}

public void challenge2 () {
    System.out.println("--- 3 TRANSAKSI TERAKHIR ---");
    System.out.println("No. Rekening : " + nomorRekening);

    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Belum ada transaksi pada rekening
ini");
    } else {
        int totalTransaksi = riwayatTransaksi.size();
        int mulai = Math.max(0, totalTransaksi - 3); // cegah
index negatif kalau transaksi < 3

        for (int i = mulai; i < totalTransaksi; i++) {
            Transaksi t = riwayatTransaksi.get(i);
            t.cetakDetail();
        }
    }
    System.out.println("-----");
}

```

```
}
```

1.2.3 Hasil

Output Program

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
Akun Aktif saat ini: -
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 0901
Masukkan Nama Pemilik: zaza
Masukkan Saldo Awal : 5000
Rekening atas nama zaza berhasil dibuat dengan saldo Rp5000.0

Menu Utama:
Akun Aktif saat ini: 0901 (zaza)
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 0902
Masukkan Nama Pemilik: dzhillan
Masukkan Saldo Awal : 800000
Rekening atas nama dzhillan berhasil dibuat dengan saldo Rp800000.0
```

```
Menu Utama:
Akun Aktif saat ini: 101 (zaza)
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Riwayat Transaksi (Mutasi)
7. Challenge 3
8. Tampilkan 3 Riwayat terbaru Challenge 2
0. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 101
Nama Pemilik : zaza
Saldo Akhir : Rp1500000.0
-----
```

```
Menu Utama:
Akun Aktif saat ini: 101 (zaza)
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Riwayat Transaksi (Mutasi)
7. Challenge 3
8. Tampilkan 3 Riwayat terbaru Challenge 2
0. Keluar
Pilih menu: 6
--- RIWAYAT TRANSAKSI ---
No. Rekening : 101
ID: TRX-S-1790095833359 | Jenis : Kredit | Nominal : Rp600000.0
```